

**< Sistem de interpretare automate a limbajului
semnelor utilizand tehnici de AI >**

Documentul de proiectare

Cuprins

1. Introducere	1
1.1 Scopul documentului.....	1
2. Prezentare generală și abordări de proiectare	2
2.1 Prezentare generală	2
2.2 Presupuneri/ Constrângeri/ Riscuri	2
2.2.1 Presupuneri	2
2.2.2 Constrângeri	2
2.2.3 Riscuri.....	3
3. Considerații de proiectare.....	4
3.1 Obiective și linii directoare (ghiduri)	4
3.2 Metode de dezvoltare	4
3.3 Strategii de arhitectură.....	4
4. Arhitectura Sistemului și Proiectarea Arhitecturii	5
4.1 Vedere logică	5
4.2 Arhitectură hardware.....	5
4.3 Arhitectură software	5
4.4 Arhitectura informațiilor	6
4.5 Arhitectura de comunicații interne.....	6
4.6 Diagrama de arhitectură a sistemului.....	6
5. Proiectarea sistemului.....	7
5.1 Proiectarea bazei de date	7
5.1.1 Obiecte de date și structuri de date rezultante	7
5.1.2 Fișiere și baze de date.....	7
5.2 Conversii de date	7
5.3 Interfețe utilizator	7
5.3.1 Intrări	8
5.3.2 ieșiri	8
5.4 Proiectarea interfețelor cu utilizatorul.....	8
6. Scenarii de utilizare	9
7. Proiectare de detaliu.....	10
7.1 Proiectare hardware de detaliu	10
7.2 Proiectare software de detaliu	10
7.2.1 Identificator serviciu: Landmark_Extraction_Service	10
7.2.2 Identificator serviciu: LSTM_Inference_Service.....	10
7.2.3 Identificator serviciu: Output_Manager_Service	11
7.3 Proiectare detaliată de securitate.....	11
7.4 Proiectare de detaliu pentru performanța sistemului.....	11

7.5	Proiectare detaliată a comunicațiilor interne (între componente)	12
8.	Controale pentru verificarea integrității sistemului	13
Anexa A:	Gestiunea modificărilor documentului	14
Anexa B:	Acronime	15
Anexa C	Documente la care se face referire.....	16

1. Introducere

Prezentul document furnizează informațiile de identificare și specificațiile tehnice detaliate pentru sistemul software p DeafEar. Acest proiect de dezvoltare reprezintă un sistem informatic bazat pe inteligență artificială , menit să faciliteze comunicarea și incluziunea socială a persoanelor cu deficiențe de auz în mediul digital (videoconferințe, telemuncă, e-learning). Evoluția așteptată a acestui document este de a servi drept referință centrală pe tot parcursul fazelor de codare, testare și implementare ale proiectului de licență, suferind modificări minore doar în faza de optimizare a codului.

În ceea ce privește securitatea și confidențialitatea, sistemul DeafEar a fost proiectat cu o arhitectură "Privacy-by-Design". Toată procesarea vizuală și biometrică a utilizatorului se realizează exclusiv local. Datele nu părăsesc niciodată sistemul de operare gazdă, eliminând riscul de interceptare a fluxurilor video sau de furt de date biometrice.

Acest Document de Proiectare a Sistemului descrie modul în care cerințele funcționale și non-funcționale înregistrate în Documentul de Cerințe (SRS) se transformă în specificații de proiectare a sistemului, la un nivel profund tehnic, pe baza cărora se construiește arhitectura DeafEar. Documentul detaliază atât proiectarea de nivel înalt, cât și specificațiile detaliate pentru fiecare componentă software, comunicațiile interne, hardware, controalele de integritate și interfețele externe.

1.1 Scopul documentului

Prin acest livrabil se documentează și se urmăresc informațiile necesare pentru a defini eficient arhitectura și designul sistemului DeafEar, în scopul de a oferi echipei de dezvoltare (în acest caz, autorului lucrării de licență) o îndrumare clară asupra arhitecturii software ce urmează să fie dezvoltată.

Documentele de proiectare sunt produse incremental și iterativ pe parcursul ciclului de viață al dezvoltării sistemului, în funcție de circumstanțele particulare ale proiectului și de metodologia hibridă de dezvoltare utilizată. Publicul țintă este format din studentul dezvoltator, profesorul coordonator (managerul de proiect) și comisia de evaluare. Anumite părți ale acestui document, cum ar fi interfața cu utilizatorul (UI), pot fi împărtășite cu asociațiile pentru persoane cu deficiențe de auz ale căror contribuții/aprobări sunt necesare pentru validarea utilității.

2. Prezentare generală și abordări de proiectare

2.1 Prezentare generală

DeafEar acționează ca un intermediar între hardware-ul de captură și aplicațiile client de conferință. Abordarea de proiectare este structurată sub forma unui Data Pipeline asincron. Imaginile sunt captate, trecute printr-un filtru de extracție a trăsăturilor biometrice (MediaPipe), convertite în vectori numerici, analizate temporal, traduse în text/audio și injectate într-un periferic virtual. Obiectivul suprem de proiectare este menținerea unei latențe scăzute prin execuție On-Edge.

2.2 Presupuneri/ Constrângeri/ Riscuri

2.2.1 Presupuneri

Pentru ca designul arhitectural propus să fie funcțional, se consideră valabile următoarele presupuneri fundamentale:

- ❖ **Hardware asociat:** Sistemul gazdă dispune de un procesor multicore cu suport pentru instrucțiuni AVX2, capabil să execute calcule vectoriale complexe.
- ❖ **Sisteme de operare:** Dezvoltarea și rularea se efectuează pe un mediu Windows 10(64-bit), unde API-urile DirectShow pentru camerele virtuale sunt stabile.
- ❖ **Mediul utilizatorului final:** Se presupune că utilizatorul va folosi aplicația într-o încăpăre cu iluminare ambientală adecvată, astfel încât senzorul camerei să nu introducă zgomot ISO excesiv.
- ❖ **Modificări probabile:** Funcționalitatea de bază va rămâne stabilă, dar vocabularul recunoscut se va extinde.

2.2.2 Constrângeri

Limitările și constrângerile globale care au un impact semnificativ asupra designului sunt:

Mediu hardware: Sistemul este constrâns să ruleze fluid pe un PC de uz casnic, fără a necesita plăci video dedicate. RAM-ul utilizat nu trebuie să depășească 1.5 - 2 GB.

Cerințe de performanță: Procesarea întregului ciclu de la captură la afișare trebuie să aibă loc în sub 50 ms pentru a menține minim 20 FPS.

Securitate: Sistemul este constrâns legal după GDPR să proceseze datele strict On-Edge, neavând voie să transmită imaginile în cloud.

Cerințe de interfață/protocol: Ieșirea video trebuie să respecte formatele standard Windows (YUY2/NV12) pentru a fi recunoscută de platformele web.

Cerințe de interoperabilitate: Arhitectura modelului neural trebuie exportată în format ONNX pentru a garanta execuția rapidă pe CPU.

2.2.3 Riscuri

- ❖ **Blocajul interfeței :** Procesarea AI poate îngheța interfața grafică din cauza limitărilor Python.
Reducere: Utilizarea arhitecturii bazate pe QThreads și cozi de mesaje.
- ❖ **Instabilitate temporală (Flickering):** Modelul AI poate oferi predicții fluctuante la fiecare cadru.
Reducere: Proiectarea unui filtru logic de Majority Voting.
- ❖ **Supraîncălzirea hardware:** Rularea AI-ului continuu poate duce la Thermal Throttling.
Reducere: Decimarea inferenței — rularea modelului LSTM o dată la 2-3 cadre.

3. Considerații de proiectare

3.1 Obiective și linii directoare (ghiduri)

Obiectivul major de proiectare este raportul optim între **Viteză și Acuratețe**. Se pune accent pe un "feel" nativ și rapid al aplicației. Liniile directoare includ respectarea standardului PEP-8 pentru codul Python. O politică de design fundamentală a fost alegerea PySide6 în loc de un framework web, motivul fiind necesitatea accesului direct la ferestrele memoriei partajate ale sistemului de operare pentru a obține o viteză maximă în manipularea matricelor video. Interfața va urma un stil "Dark Mode" pentru reducerea oboselii oculare.

3.2 Metode de dezvoltare

S-a optat pentru o metodă de dezvoltare hibridă: **Prototipare și Programare Orientată pe Obiecte**. Inițial, modelele matematice au fost prototipate în Jupyter Notebooks. Ulterior, logica a fost transpusă într-o arhitectură OOP pentru a asigura scalabilitatea. Contingențele identificate se referă la posibila incompatibilitate a librăriei de virtualizare cu anumite drivere video; planul alternativ este utilizarea interfețelor de Shared Memory direct către software-ul open source OBS Studio ca intermediar.

3.3 Strategii de arhitectură

- ❖ Utilizarea produselor/bibliotecilor: S-a decis reutilizarea MediaPipe Holistic extragerea punctelor biometrice, evitând reinventarea detecției umane și concentrând resursele pe componenta de inferență a limbajului.
- ❖ Paradigmele interfeței: Model de tip "Dashboard Minimalist", ascunzând complexitatea AI-ului în spatele unui panou simplu de control.
- ❖ Managementul memoriei: Se vor utiliza matrici statice pre-allocate și Circular Buffers pentru a evita alocarea/dealocarea dinamică a memoriei la fiecare cadru (prevenind Garbage Collection Spikes).
- ❖ Concurgență: Arhitectură multi-threading bazată pe sistemul de Signals and Slots specific framework-ului Qt, decuplând vizualul de procesarea matematică.

4. Arhitectura Sistemului și Proiectarea Arhitecturii

Aplicația interacționează cu hardware-ul pentru captură, cu propriile module pentru procesare și cu driverele OS-ului pentru a expune datele finale.

Responsabilitățile sunt partiționate clar: Achiziție, Viziune, Inferență, și Sinteză.

4.1 Vedere logică

Din punct de vedere logic, arhitectura respectă tiparul **Pipes and Filters** suprapus pe **MVC**. Datele trec succesiv prin filtre matematice. Modelul menține ponderile AI, View-ul randează matricea finală, iar Controller-ul assemblează pipeline-ul. Modulul de inferență este izolat, comunicând doar prin vectori de intrare și string-uri de ieșire.

4.2 Arhitectură hardware

Sistemul este complet centralizat, rulând exclusiv local.

- ❖ **Componente:** Un singur PC/Laptop, o cameră web conectată via USB, un microfon virtual și un monitor.
- ❖ **Estimări:** Procesorul va fi utilizat la ~35% din capacitate, 1.5 GB RAM pentru operare, 50 MB stocare pentru aplicație și model. Nu sunt necesare conexiuni de rețea .

4.3 Arhitectură software

Limbaj de programare: Python 3.10.

Componente logice (Biblioteci): NumPy (calcul matriceal), OpenCV (procesare BGR/RGB), MediaPipe (detectie landmarks), ONNX Runtime (execuția modelului PySide6 (Prezentare), pytsx3 (Sinteză audio), pyvirtualcam (Comunicare). Nu necesită licențe comerciale.

Structură ierarhică:

- ❖ core_engine.py (Managerul fluxului de date)
- ❖ vision_module.py (Extracție)
- ❖ inference_module.py (AI)
- ❖ ui_layer.py (Prezentare)
- ❖ virtual_bridge.py (Export) Interacțiunile se fac strict prin transfer de matrici NumPy.
- ❖ Părțile preexistente (cum ar fi driverul OBS) sunt interogate doar pentru a le trimite date.

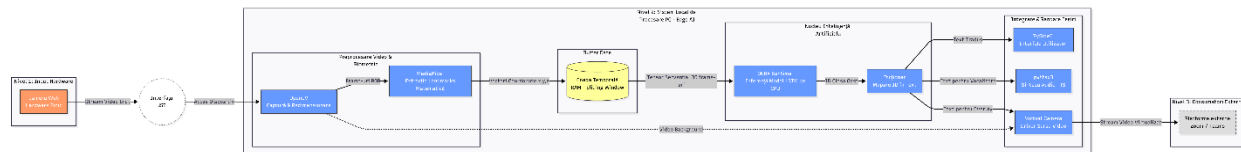
4.4 Arhitectura informațiilor

Informațiile stocate sunt minime și nu conțin date cu caracter sensibil (PII - Personally Identifiable Information). Datele furnizate manual sunt doar fișierele de configurare. Informația dinamică (fluxul video) provine exclusiv din interfața hardware a camerei și este volatilă.

4.5 Arhitectura de comunicații interne

Fiind un sistem rulat pe o singură mașină, rețeaua de comunicații interne nu se aplică în sens tradițional. Comunicația componentelor are loc via **Inter-Process Communication (IPC)** și **Memorie Partajată**. Fluxul de comunicație implică transferul pointerilor de memorie între thread-ul de Python și interfața DirectShow a Windows-ului. Lățimea de bandă necesară este lățimea magistralei PCIe/RAM locale pentru a muta cadre necomprimate de 640x480 pixeli la 30 FPS.

4.6 Diagrama de arhitectură a sistemului



5. Proiectarea sistemului

5.1 Proiectarea bazei de date

DeafEar utilizează exclusiv fișiere non-DBMS. Nu există niciun motor SQL (MySQL, Oracle) asociat. Toate stocările sunt de tip fișiere locale (flat-files) pentru performanță extremă și portabilitate.

5.1.1 Obiecte de date și structuri de date rezultante

- ❖ **Frame_RGB**: Structură de date de tip matrice , folosită pentru stocarea imaginii vizuale.
- ❖ **Landmarks_Vector**: Structură array 1D de lungime 258, stocând numere reale ce reprezintă coordonatele corpului uman normalizate spațial. Transmisă de la modulul vizual la modulul AI.

5.1.2 Fișiere și baze de date

Modelul fizic stochează datele în directorul root al aplicației.

5.1.2.1 Fișiere non-DBMS

- ❖ **model_lstm.onnx**: Fișier binar read-only. Conține graful rețelei neurale. Citit o singură dată la inițializare de modulul de inferență.
- ❖ **dictionary.json**: Fișier text JSON. Citit la inițializare. Conține perechi cheie-valoare (ex: "1": "Bună ziua"). Mapează ieșirea AI cu limba umană. Actualizat doar la lansarea unor noi versiuni de vocabular.
- ❖ **config.json**: Fișier text JSON de configurare. Utilizat pentru intrare și ieșire. Scris de interfața UI când utilizatorul modifică o setare, citit la start-up. Stochează indici de rezoluție, camera preferată și threshold-ul AI.

5.2 Conversii de date

- ❖ **Formatul de culoare**: Conversie masivă BGR -> RGB (pentru analiză) -> YUY2 (pentru export către driverul video virtual).
- ❖ **Normalizarea geometrică**: Coordonatele absolute în pixeli sunt convertite în valori relative cuprinse între [0, 1] prin scăderea originii (centrul pieptului) și împărțirea la o distanță constantă (lățimea umerilor), asigurând invarianța datelor.

5.3 Interfețe utilizator

Utilizatorul principal vizat este persoana cu deficiențe de auz (Utilizator Operațional), estimat la o singură instanță concurentă per dispozitiv. Rolul acestuia este să inițieze programul, să seteze preferințele și să efectueze gesturile.

5.3.1 Intrări

Intrări video: Sursa principală de date, preluată automat din senzorul optic selectat.

Intrări GUI: Utilizatorul oferă comenzi prin mouse/tastatură pe ecranul principal.

Elementele de date includ:

- ❖ Camera_Selector (ComboBox): Alege hardware-ul (0, 1, 2).
- ❖ Confidence_Slider (Glisor): Valori stricte între 0.50 și 0.95. Previne supra-editarea.
- ❖ Mute_Audio_Toggle (Checkbox): Controlează boolean starea motorului TTS.

5.3.2 Ieșiri

Ieșirea grafică internă (Dashboard): Un ecran care afișează imaginea live a utilizatorului, pe care este suprapus scheletul biometric (pentru validarea poziționării în cadru). Conține un panou cu textul tradus în timp real.

Ieșirea video externă (Virtual Cam): Flux video necomprimat ce conține DOAR textul tradus și utilizatorul (fără schelet sau butoanele aplicației), servit ca sursă pentru aplicații externe.

5.4 Proiectarea interfețelor cu utilizatorul

Interfața este dezvoltată în PySide6, folosind un layout tip Grid. Zona principală conține elementul de vizualizare video. O bandă lată inferioară prezintă textul tradus. Meniul lateral oferă setările descrise la Intrări, cu mesaje clare de avertizare în caz de „Cameră nedetectată” sau „Mediu prea întunecat”.

6. Scenarii de utilizare

Scenariul 1: Desfășurarea unei ședințe pe Microsoft Teams

1. **Stimul/Eveniment:** Utilizatorul deschide DeafEar.
2. **Acțiune:** Sistemul inițializează camera hardware și creează camera virtuală "DeafEar Cam". Interfața afișează imaginea live.
3. **Interacțiune externă:** Utilizatorul deschide Teams, alege "DeafEar Cam" la setările video și "Virtual Audio Cable" la microfon.
4. **Operațional:** Utilizatorul realizează gestul pentru „Salut” în fața laptopului.
5. **Procesare internă:** Sistemul capturează, analizează 30 de cadre, deduce cuvântul, îl trimite la motorul audio (care pronunță "Salut") și îl suprapune ca text pe ieșirea video.
6. **Efect negativ prevenit:** Dacă utilizatorul iese din cadru, sistemul nu mai traduce mișcările de fundal (scaun, uși), oprind inferența.

7. Proiectare de detaliu

7.1 Proiectare hardware de detaliu

DeafEar nu necesită asamblare hardware customizată, integrând articole COTS:

- **Camera Web:** Conectare prin USB Type-A sau Type-C. Specificații: Senzor CMOS, suport UVC obligatoriu, lentilă cu unghi de captură minim de 90 de grade pentru a prinde ambele mâini extinse. Fără cerințe de impedanță specială, alimentată direct din portul USB .
- **Cerințe procesor:** Minim arhitectură x86_64, cu suport pentru setul de instrucțiuni vectoriale AVX2, pentru a procesa tensori ONNX eficient.

7.2 Proiectare software de detaliu

7.2.1 Identificator serviciu: Landmark_Extraction_Service

- ❖ **Clasificare:** Serviciu de procesare date vizuale.
- ❖ **Definiție:** Extrage coordonatele tridimensionale ale articulațiilor umane din cadrele RGB.
- ❖ **Cerințe:** Viteză execuție < 25ms. Toleranță la zgomot vizual.
- ❖ **Structuri de date interne:** Matrice 1D pentru stocarea coordonatelor normalizate.
- ❖ **Constrângeri:** Depinde de calitatea iluminării. Valori de intrare strict de tip ndarray (640x480x3).
- ❖ **Compoziție:** Utilizează librăria MediaPipe Holistic.
- ❖ **Utilizări/Interacțiuni:** Consumă date de la Video_Capture_Service. Oferă date către LSTM_Inference_Service.
- ❖ **Procesare:** Algoritmul identifică regiunea de interes (Bounding Box), extrage punctele, le translatează față de centrul umerilor și le scalează pe distanța umerilor (normalizare spațială).
- ❖ **Interfețe/Exporturi:** Metoda publică process_frame(frame) -> ndarray.

7.2.2 Identificator serviciu: LSTM_Inference_Service

- ❖ **Clasificare:** Serviciu de aplicație (Inteligență Artificială).
- ❖ **Definiție:** Analizează secvențele temporale de date pentru a deduce sensul gestului.
- ❖ **Cerințe:** Timp de răspuns < 10ms (decimat).
- ❖ **Structuri de date interne:** O coadă circulară collections.deque(maxlen=30) pentru menținerea contextului temporal al ultimelor 30 de cadre.
- ❖ **Constrângeri:** Necesită inițializarea prealabilă a onnxruntime.InferenceSession. Intrările trebuie să respecte strict dimensiunea tensorului modelului.

- ❖ **Utilizări/Interacțiuni:** Primește vectori de la Landmark_Extraction_Service. Semnalează Output_Manager_Service.
- ❖ **Procesare:** Când coada de 30 de cadre este plină, rulează session.run() pe tensor. Aplică funcția Softmax pe ieșiri. Implementează algoritmul de **Vot Majoritar**: dacă clasa dominantă se repetă în 3 din ultimele 5 inferențe și are confidence > threshold, o declară validă.
- ❖ **Interfețe/Exporturi:** Eveniment on_gesture_recognized(string gesture_name).

7.2.3 Identificator serviciu: Output_Manager_Service

- ❖ **Clasificare:** Serviciu de prezentare și export date.
- ❖ **Definiție:** Suprapune grafic textul și vocalizează traducerea.
- ❖ **Cerințe:** Funcționare asincronă pentru a nu bloca conducta video.
- ❖ **Structuri de date interne:** O coadă de tip String pentru mesajele audio de reprodus.
- ❖ **Procesare:** Algoritmul preia imaginea de la captură, aplică cv2.putText cu textul primit de la inferență. În paralel, un thread preia cuvântul și apelează pyttsx3.engine.say().
- ❖ **Interfețe/Exporturi:** Interfațează cu DirectShow prin pyvirtualcam.

7.3 Proiectare detaliată de securitate

Sistemul rulează ca o aplicație desktop complet independentă (Offline), prin urmare expunerea la vectorii de atac rețelistic este nulă.

- ❖ **Autentificare și Autorizare:** Gestionate de OS. Aplicația va rula exclusiv în Sandbox-ul utilizatorului logat.
- ❖ **Auditare:** Log-urile se scriu local, dar nu conțin sub nicio formă cadre video, puncte biometrice sau traduceri. Se loghează doar indicatori de performanță hardware.
- ❖ **Criptare / Porturi:** Niciun port de rețea (TCP/UDP) nu este alocat. Modelul ONNX este stocat binar read-only.

7.4 Proiectare de detaliu pentru performanța sistemului

Performanța reprezintă pilonul de bază al sistemului în timp real.

- ❖ **Cerințe/Estimări de capacitate:** Sistemul trebuie să prelucreze cadre pe secundă continuu.
- ❖ **Așteptări de performanță:** Latență totală end-to-end de sub 800 milisecunde de la mișcarea mâinii la apariția textului.
- ❖ **Proiectare de performanță:** Arhitectura software utilizează un algoritm de Frame Skipping. Modelul de inferență, cel mai scump computațional, este rulat doar la fiecare 2 cadre. Celelalte cadre folosesc predicția anterioară. Astfel se dublează capacitatea efectivă a sistemului pe același hardware.

- ❖ **Punct unic de eșec:** Cablul USB al camerei web. Designul include un sistem de watchdog care re-inițializează conexiunea USB automat la detectarea frame-urilor nule, menținând camera virtuală activă pe fundal.

7.5 Proiectare detaliată a comunicațiilor interne (între componente)

Deoarece sistemul rulează pe un singur computer (fără LAN/WAN), comunicațiile interne presupun exclusiv schimburi de memorie (Memory Mapped I/O).

- ❖ **Sincronizare și control:** Comunicația inter-thread se face utilizând obiecte de tip Signal/Slot în Qt.
- ❖ **Formatul datelor schimbate:** Transferul video către Virtual Cam folosește formatul RAW de tip YUY2 (YUV 4:2:2). Direcția fluxului de date: Python Process > RAM Buffer > OS DirectShow Driver > Third-Party App.

8. Controale pentru verificarea integrității sistemului

Specificațiile de proiectare includ mecanisme robuste de control intern pentru garantarea stabilității:

- ❖ **Tabele standard:** leșirea rețelei neurale este restrânsă doar la valorile cunoscute mapate în dicționarul de semne (JSON). Validarea respinge orice index necunoscut
- ❖ **Procese de verificare la inițializare:** Verificarea hash-ului fișierului .onnx la start-up pentru a preveni alterarea fișierului de către softuri malware.
- ❖ **Securitate internă memorie:** Datele critice (fluxurile video din buffer) sunt suprascrise secvențial, fără păstrarea referințelor, permițând Garbage Collector-ului să șteargă imediat rămășițele, asigurând că memoria RAM nu este saturată .

Anexa A: Gestiunea modificărilor documentului

Instrucțiuni: Furnizați informații despre modul în care dezvoltarea și distribuția documentului va fi controlată și urmărită. Utilizați tabelul de mai jos pentru a furniza numărul de versiune, data versiunii, autorul/deținătorul versiunii și o scurtă descriere a motivului pentru crearea versiunii revizuite.

Tabel 1 – Înregistrarea modificărilor asupra documentului curent

versiune	Data	Autorul/Deținătorul	Descriere
<1.0>	<14/01/2026>	<Eusebiu Timpu>	< Inițierea documentului. Definirea arhitecturii generale. >
<1.3>	<18/03/2026>	<Eusebiu Timpu>	< Detalierea designului hardware și includerea tehnologiilor MediaPipe și ONNX. >

Anexa B: Acronime

Tabel 2 - Acronime

Acronim	Forma completă
AI / ML	Artificial Intelligence / Machine Learning (Inteligență Artificială / Învățare Automată)
API	Application Programming Interface (Interfață de Programare a Aplicațiilor)
COTS	Commercial Off-The-Shelf (Produse comerciale de serie)
DBMS	Database Management System (Sistem de Gestiune a Bazelor de Date)
GDPR	General Data Protection Regulation (Regulamentul General privind Protecția Datelor)
GUI	Graphical User Interface (Interfață Grafică cu Utilizatorul)
IPC	Inter-Process Communication (Comunicații inter-proces)
LSR	Limbajul Semnelor Românesc
LSTM	Long Short-Term Memory (Arhitectură de Rețea Neurală Recurentă)
MVC	Model-View-Controller (Paradigmă arhitecturală software)
ONNX	Open Neural Network Exchange (Format deschis pentru modele AI)
SRS	Software Requirements Specification (Document de Specificații a Cerințelor)
TTS	Text-To-Speech (Conversie Text în Voce)

Anexa C Documente la care se face referire

Tabel 3 – Documente la care se face referire

Nume document	Locație sau URL	Data emiter document
< Documentul de specificare a cerințelor (SRS) - DeafEar >	< Arhiva internă a proiectului >	< 18/03/2026>
< MediaPipe Holistic Framework Documentation >	< https://developers.google.com/mediapipe >	Referință Externă
< ONNX Runtime Architecture Documentation >	< https://onnxruntime.ai/docs/ >	Referință Externă